COMP 380
Lab 4 — Parallel Password Cracking with MPI

You have been diligently working on new and different ways to multiply two square matrices together for your parallel programming course when you notice that a new email is in your mailbox with a curious subject: ‘‘`PLANS FOR WORLD DOMINATION!!!`’’. Admittedly, email doesn't seem like a very good way to distribute such plans, but luckily the sender has encrypted the plans using the DES encryption algorithm.

In order to save the world as we know it, you decide to apply your parallel programming skills and decrypt the messages before the world domination deadline. If only you could submit the solutions described below to Moodle by the deadline, the world will be saved!

You decide that UPC is too much trouble and opt for a solution using MPI. Having a world-class supercomputer available to you, you decide to use the `lotus` cluster to assist you in the code breaking efforts. Luckily you can find documentation on how to use it online and via Moodle.

The email you receive has a ZIP file with a set of encrypted files and, oddly, an encrypted password entry. You suspect that due to the semi-obvious nature of the email, the password is probably a short one — perhaps five or fewer characters (printable, ASCII).

There are two components to this assignment. First, you will write a parallel program to search for a password (unique to you) that can be used to decrypt a secret message. Second, you will use a second password (shared amongst the class) to run experiments measuring the parallel performance of your solution.

Password Cracking

You manage to find a sample program on the Internet (which has been archived on Moodle) that demonstrates how to work with passwords and encryption. A *password* can be used to verify one's identity. How can we check passwords without storing a clear text version of the password on the authentication server? Enter the *one-way hash function*. This is a function that takes a plaintext string and hashes it into a string of seemingly random garbage. This function only works one way: when you type in your password, the hashed garbage can be compared against a stored version of the hashed garbage. If they match, then the original passwords must be the same. The function is one-way in that one cannot take the garbage string and work backwards to produce the original string. The program you find demonstrates how to generate this hashed/encrypted string using the OpenSSL library. To crack the password you will have to try all possible combinations until one of your guesses matches the hashed garbage string.

Once you find the plaintext version of the password, you will still need to decrypt the secret message using the DES encryption algorithm. The program also shows how to use the OpenSSL library to encrypt and decrypt a block of data using DES, with the clear text key that you found by cracking the password.

Your program must use the MPI programming API to perform a parallel search of all possible combinations of letters, numbers, and symbols, stopping when it has found a string that hashes to the password found in your email. Once you have found the password, you should decrypt the file in the ZIP file, using the DES algorithm.

Parallel Performance

You notice that one of the files is named “`passwords.des`”, which catches your eye. This file recently made headlines in a dump from Alfred Wikileeks Snowman, where it was released with a text file only containing the string: `wDqVEHozzDT1E`. This hash is shared by the whole class, and you will use it to measure how well your parallel search scales.

For the timing experiments you must run your search over the *entire* keyspace, to completion, *without*

stopping early when the password is found. (A clean way to do this is a command-line switch that disables the "found" short-circuit, leaving the rest of your program unchanged.) This matters a great deal: if you stop as soon as some process finds the password, your running time depends mostly on *where* in the search space the password happens to sit, not on how well your code parallelizes. Two students with identical programs would report wildly different speedups, and the same program on different processor counts would not even be comparable to itself. By always searching the full space, every run performs exactly the same amount of work, so differences in running time reflect only parallel overhead and load balance — which is what we actually want to measure.

Write up a short report for your professor with graphs of execution time, speedup, and efficiency for the full-keyspace search on the following processor counts: 1, 2, 4, 8, 16, 32, 48, 96, 192, 384, 768, 960. Each `lotus` compute node has 48 cores, so the counts from 48 upward correspond to whole nodes (48 is one node, 960 is twenty nodes). In your analysis, account for any departure from ideal (linear) speedup. Three effects are worth discussing: the cost of the collective communication you use to coordinate the processes, the load imbalance that appears when the number of search tasks does not divide evenly among the processes, and any change in scaling you observe once the job grows past a single node and must communicate over the network rather than within shared memory.

Building and Running on `lotus`

Load the toolchain and build with `make`. The `OpenSSL/3` module supplies the OpenSSL headers used for the DES routines.

```
module unload openshmem ucx
module load openmpi5 OpenSSL/3
make

# Part 1: crack your password and decrypt your message
mpirun -n 8 ./lab4

# Part 2: full-keyspace sweep (the switch you add to disable the early stop)
mpirun -n 8 ./lab4 --sweep
```

For the larger processor counts, submit a batch job with `sbatch`/`srun` rather than running on the login node, and request whole nodes (48 cores each).

Quick! To your editor! And your compiler! Save the world before it's too late!